

Guia de Traceroute

Guia de Traceroute

Este guia contém todo o conhecimento necessário para completar os Projetos 1 e 2.

Este guia não depende de nenhuma aula anterior, embora as Aulas 1–3 (aulas introdutórias) possam ser úteis.

1. Contexto de Redes

1.1. Protocolos

Nesta disciplina, grande parte do nosso foco estará em **protocolos**, que especificam como as máquinas trocam comunicação. Qual é o formato das mensagens que elas trocam, e como respondem a essas mensagens?

Todos que usam o protocolo precisam concordar sobre como os dados são formatados e o que devem fazer em resposta a diferentes mensagens.

Como analogia, considere um protocolo para fazer uma pergunta em sala de aula. Primeiro, você deve levantar a mão. Você deve esperar o professor te dar a palavra antes de fazer a pergunta. Se o professor não te chamar, você pode dizer “Com licença!” para chamar a atenção.

Aqui está um exemplo de protocolo. Alice e Bob dizem olá um ao outro, depois Alice solicita um arquivo, e Bob responde com o arquivo. Para definir esse protocolo, precisamos definir a sintaxe (por exemplo, como escrever “me dê este arquivo” em 1s e 0s) e a semântica (por exemplo, Alice precisa receber um olá de Bob antes de solicitar um arquivo).

Diferentes protocolos são projetados para diferentes necessidades. Por exemplo, se Alice precisa obter o arquivo o mais rápido possível, poderíamos projetar um protocolo sem as mensagens iniciais de saudação.

Projetar um bom protocolo pode ser mais difícil do que parece! Também podemos precisar levar em conta casos extremos, bugs e comportamento malicioso. Por exemplo, e se Alice solicitar um arquivo e Bob responder com um olá? Como Alice deveria reagir?

1.2. Construindo a Internet

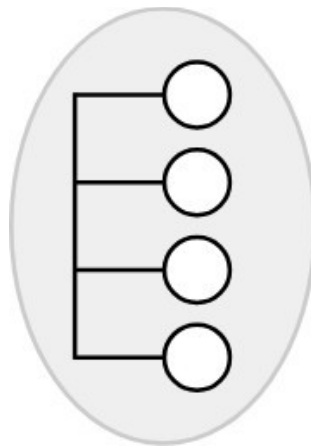
Para projetar a Internet, vamos pensar no sistema postal como analogia. Como projetamos o sistema postal?

Bloco de construção 1: precisamos de alguma tecnologia física para mover dados através do espaço. Poderia ser um carteiro caminhando, um caminhão de correio, um avião, o Pony Express, pombos-correio, etc.

Da mesma forma, na Internet, precisamos de alguma tecnologia física para mover bits através do espaço. Poderia ser tensões em um fio elétrico, sinais de luz em uma fibra óptica, ondas de rádio sem fio, etc.

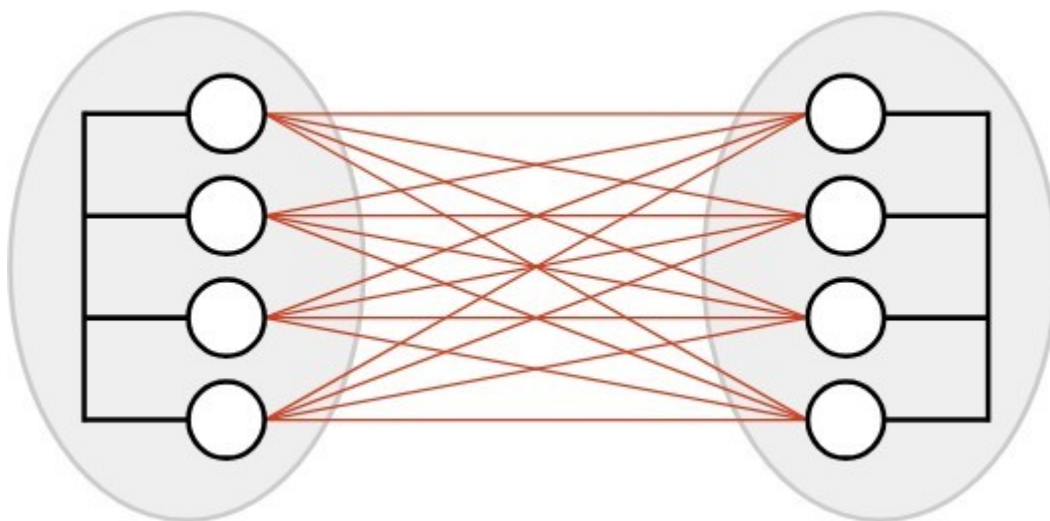
Bloco de construção 2: usando nossa tecnologia física, podemos criar enlaces para conectar todos na cidade local.

Da mesma forma, na Internet, podemos usar nossa tecnologia física para conectar todos em uma área local.



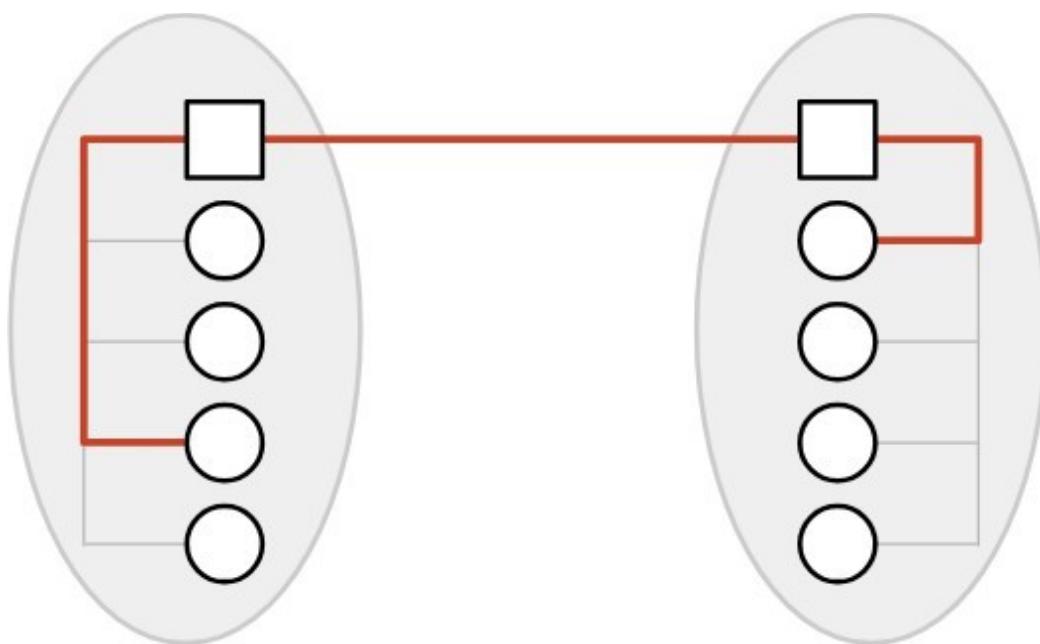
Rede local (LAN) conectando computadores próximos.

Bloco de construção 3: e se quiséssemos conectar casas em cidades diferentes? Poderíamos criar enlaces diretos entre cada par de casas, mas isso é caro.



Conectar casas diretamente por meio de vários enlaces é caro.

Em vez disso, poderíamos introduzir uma agência dos correios em cada cidade. Agora, só precisamos de um único enlace entre as duas cidades para conectar as duas agências.



Um roteador (agência dos correios) conecta duas redes locais.

Se eu estou em Oakland e quero enviar correspondência para você em Berkeley: eu envio minha carta para a agência dos correios de Oakland. A agência de Oakland encaminha a carta para a agência de Berkeley. Por fim, a agência de Berkeley encaminha a carta até você.

Com agências dos correios suficientes, podemos conectar todas as cidades do mundo!

1.3. Propriedades da Internet

Note que nosso design decompôs o sistema em camadas de abstração. Cada camada depende de serviços da camada abaixo. Cada camada, então, fornece seus serviços à camada acima.

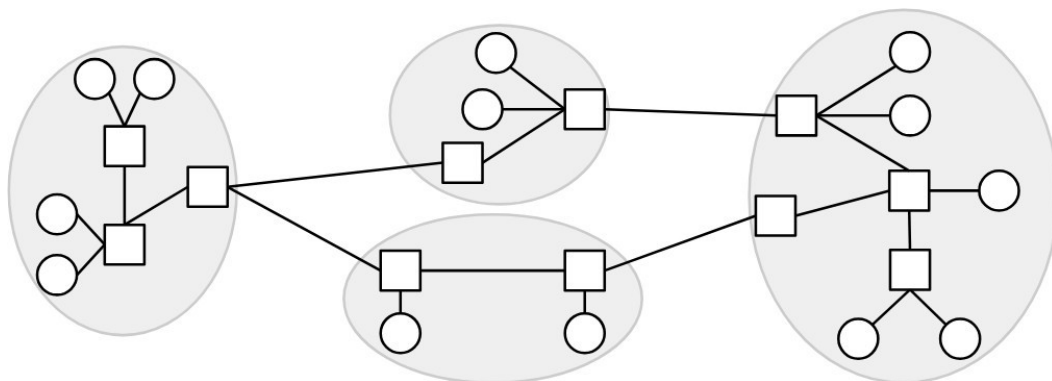
Camada 1 (Física) move bits através do espaço.

Camada 2 (Enlace) cria enlaces em uma rede local. A Camada 2 depende da tecnologia que construímos na Camada 1.

Camada 3 (Internet) conecta redes locais para formar a Internet. A Camada 3 depende das redes locais que construímos na Camada 2.

A Internet é frequentemente chamada de **“rede de redes”**. Cada operadora administra sua própria rede local (pense nisso como uma pequena cidade). Depois, as redes locais se

conectam umas às outras para formar a Internet.



Várias redes locais interligadas formam a Internet.

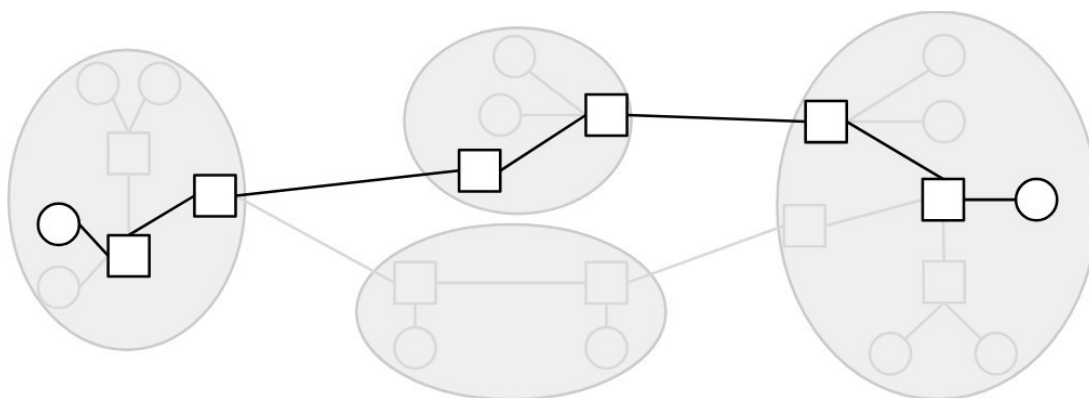
Existem dois tipos de máquinas na Internet:

Hosts finais são as máquinas que realmente conversam entre si (por exemplo, seu laptop, seu celular, o servidor web do Google). Na analogia, são as casas. Normalmente desenhamos hosts finais como círculos.

Roteadores (também chamados de switches) são as máquinas que recebem mensagens e as encaminham em direção ao destino. Na analogia, são as agências dos correios. Normalmente desenhamos roteadores como quadrados.

No nosso modelo, um enlace conecta duas máquinas. Porém, a origem e o destino não estão necessariamente conectados diretamente por um enlace.

Em vez disso, uma mensagem pode dar múltiplos **saltos** (hops) até chegar ao destino. Quando um roteador recebe uma mensagem, ele precisa **encaminhá-la** (forward) para mais perto do seu destino final.



Uma mensagem dá múltiplos saltos entre roteadores até chegar ao destino.

Por enquanto, não vamos discutir como os roteadores descobrem para onde encaminhar as mensagens. Porém, note que, se você enviar várias mensagens para o mesmo destino, elas

não têm garantia de seguir o mesmo caminho.

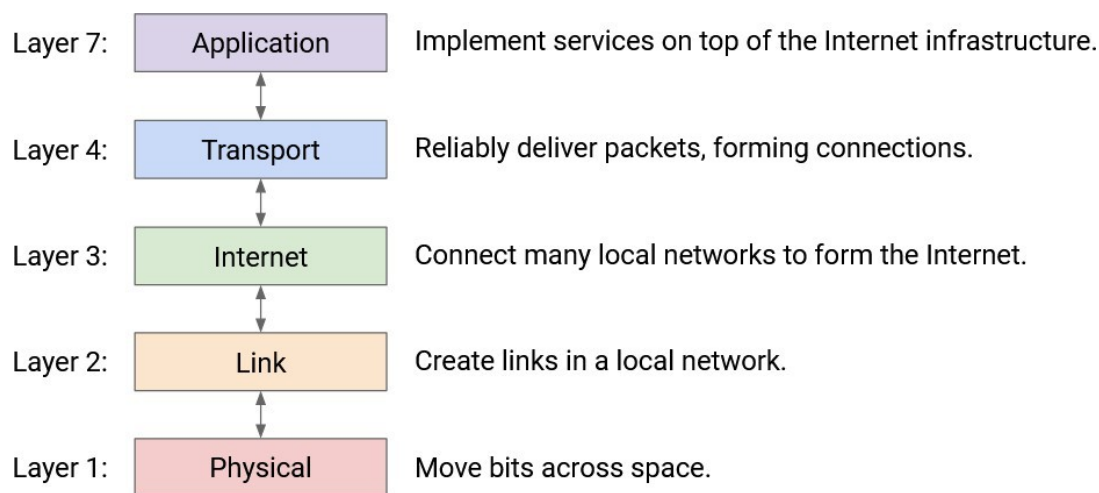
A Internet oferece apenas serviço de **melhor esforço** (best-effort). Isso significa que as mensagens podem se perder, chegar fora de ordem, ser corrompidas, etc. A rede vai tentar o seu melhor para entregar sua mensagem, mas não há garantias.

Se quisermos algo melhor que o melhor esforço, precisaremos construir mais camadas.

1.4. Camadas Superiores

Camada 4 (Transporte) adiciona recursos extras (por exemplo, reenviar mensagens perdidas) para garantir a entrega confiável. A Camada 4 também fornece um recurso chamado portas (discutido mais adiante). A Camada 4 depende da entrega global de melhor esforço que construímos na Camada 3.

Camada 7 (Aplicação) é onde construímos os programas de fato que usam a Internet. Ela depende dos recursos (confiabilidade, portas) que construímos na Camada 4.

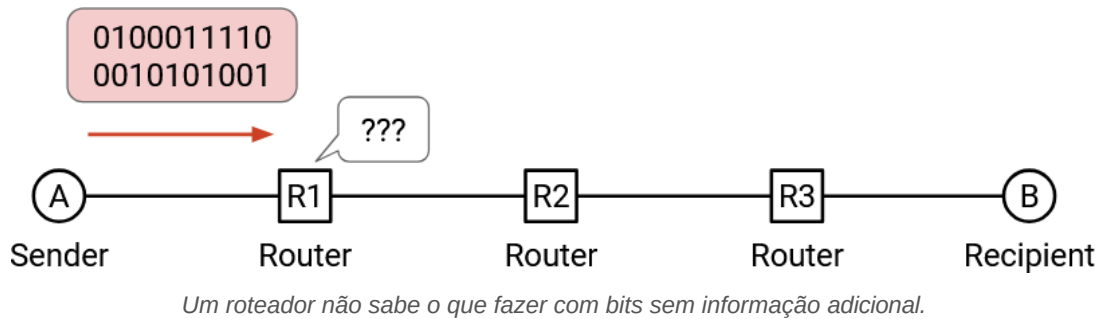


Neste projeto, nosso foco principal será na Camada 3 (Internet) e na Camada 4 (Transporte).

1.5. Cabeçalhos

Suponha que A queira enviar uma mensagem para B.

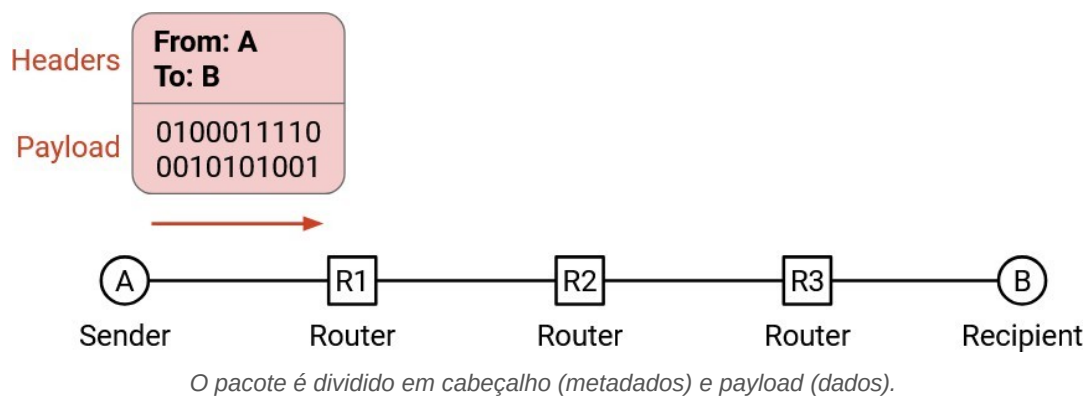
A pega os bits da mensagem e os transmite para o próximo roteador ao longo do caminho. O roteador não tem ideia do que fazer com esses pacotes!



Quando enviamos uma mensagem, precisamos incluir alguns metadados, para que os roteadores intermediários e o destinatário saibam o que fazer com os bits que recebem.

Na analogia postal: precisamos colocar a carta em um envelope. O envelope descreve o que fazer com a carta.

A unidade principal de dados na Internet é um **pacote**. Um pacote contém um **cabeçalho** com metadados sobre a mensagem, seguido de um **payload** (carga útil) contendo a própria mensagem.



Quais metadados vão no cabeçalho?

Precisamos incluir o endereço de destino, para que o pacote possa ser entregue ao destinatário correto.

Frequentemente também incluimos o endereço de origem, caso o destinatário queira enviar respostas de volta.

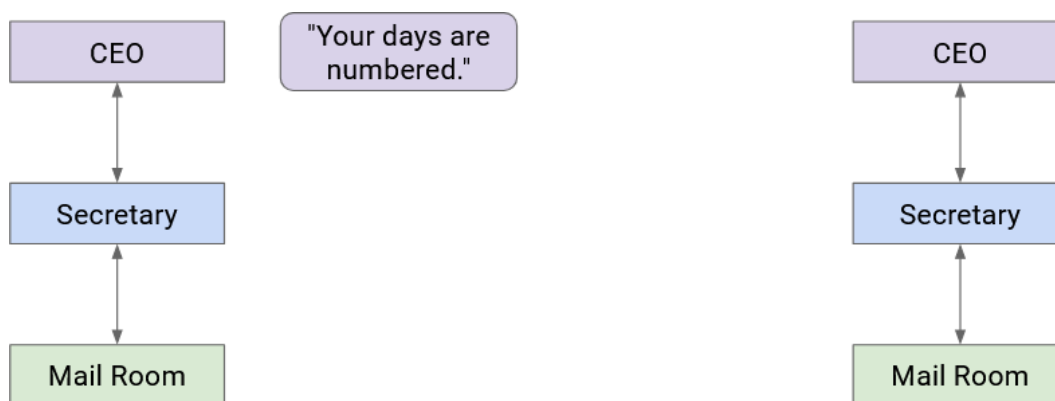
Para este projeto, não discutiremos de onde vêm esses endereços. Vamos assumir magicamente que todas as máquinas têm endereços IP únicos, escritos como quatro números separados por pontos (por exemplo, 3.3.3.3).

As máquinas também têm nomes legíveis por humanos, como **www.berkeley.edu**, e você pode vê-los no projeto às vezes. Para este projeto, não discutiremos de onde vêm esses

nomes. Vamos assumir magicamente que todos conseguem converter entre nomes e endereços conforme necessário (dê uma olhada em DNS se quiser um adiantamento).

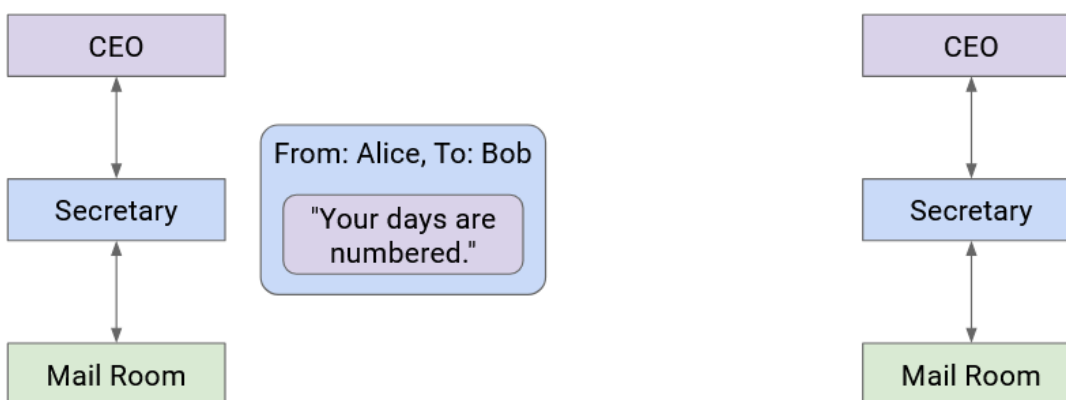
1.6. Múltiplos Cabeçalhos

Vamos voltar brevemente à analogia postal. Suponha que o chefe da Empresa A queira escrever uma carta para o chefe da Empresa B. Como a mensagem é enviada?



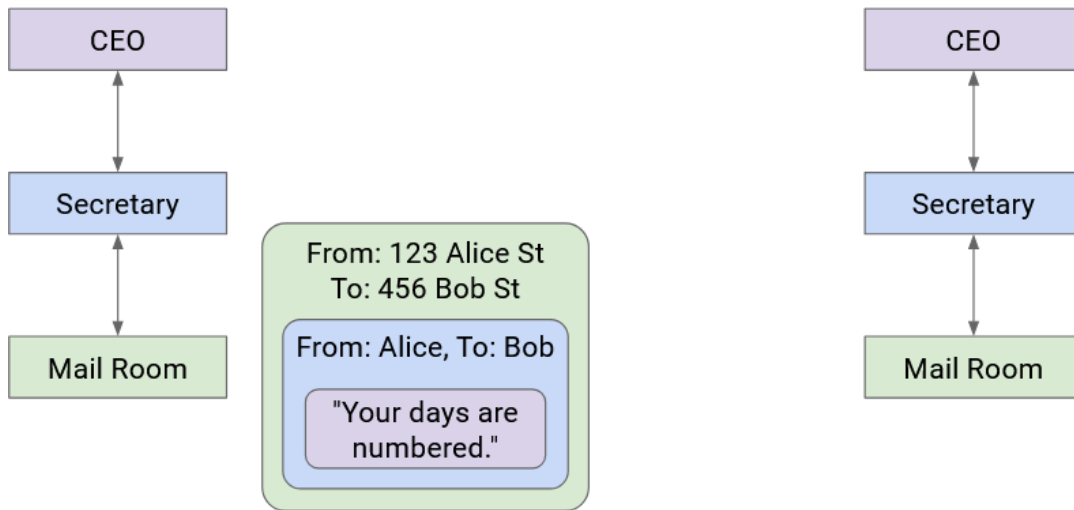
O chefe da Empresa A entrega a carta à secretária.

O chefe da Empresa A dobra a carta e a entrega à sua secretária. Então, a secretária coloca a carta em um envelope com o nome completo do chefe da Empresa B.



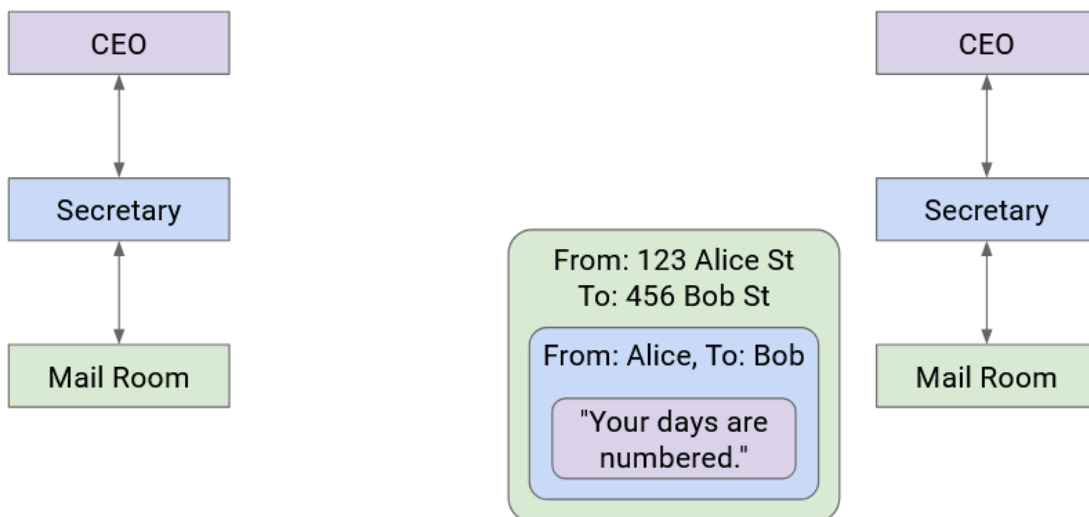
A secretária coloca a carta em um envelope endereçado ao chefe da Empresa B.

A secretária entrega essa carta para o setor de correspondência. O funcionário dos correios coloca a carta em uma caixa com o endereço da rua da Empresa B e coloca o pacote em um caminhão de entrega.



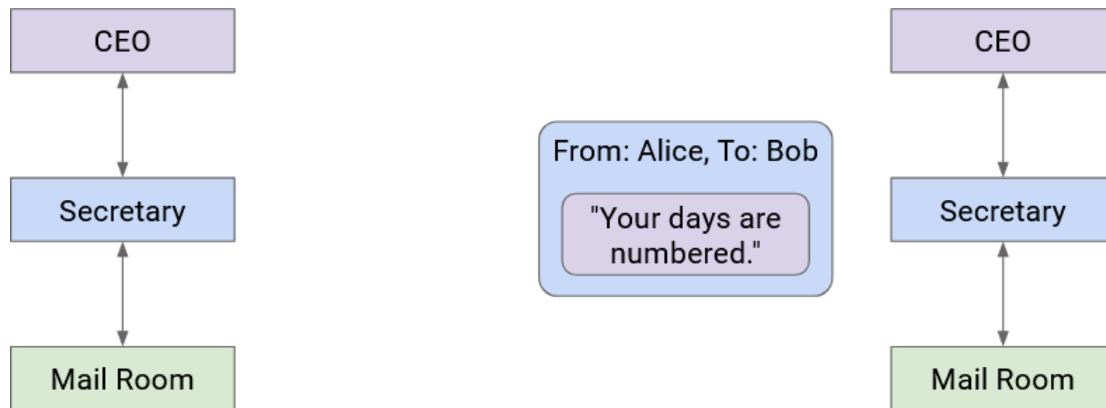
O envelope é colocado dentro de uma caixa endereçada à Empresa B.

Neste ponto, a carta em si está envolta em múltiplas camadas de informações de identificação (envelope, caixa). A empresa de entregas envia a carta para a Empresa B (possivelmente através de vários caminhões, aviões, carteiros, etc.).



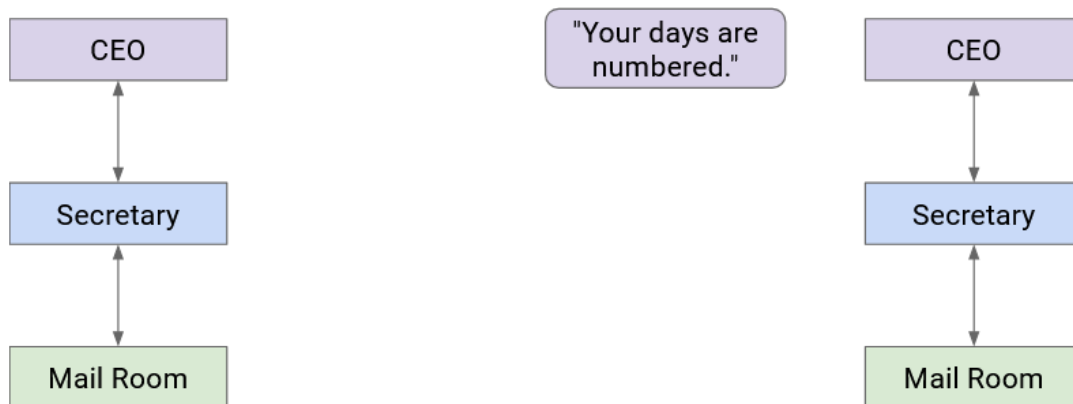
A caixa viaja até a Empresa B através da transportadora.

Quando a carta chega à Empresa B, o setor de correspondência remove a caixa e passa o envelope para a secretária.



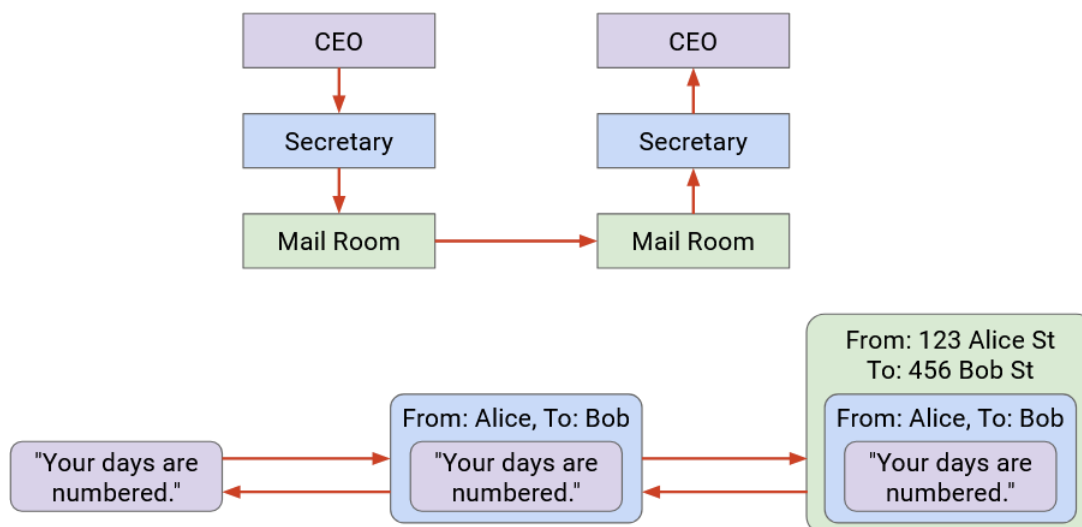
O setor de correspondência da Empresa B remove a caixa, revelando o envelope.

Então, a secretária vê o nome do chefe no envelope, remove o envelope e entrega a carta ao chefe da Empresa B.



A secretária da Empresa B remove o envelope e entrega a carta ao chefe.

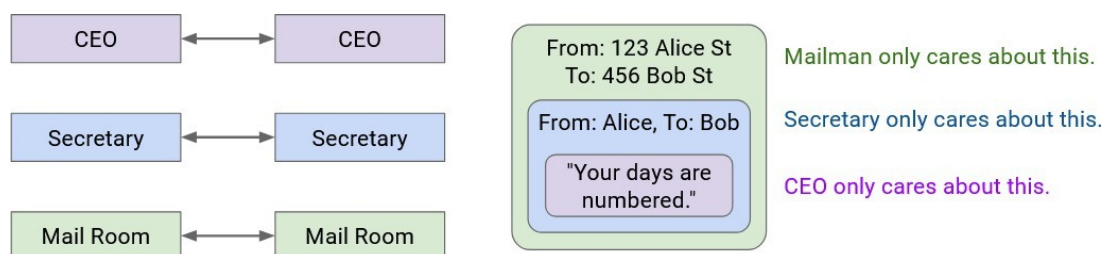
Note que, à medida que descíamos para camadas de abstração mais baixas, envolvemos mais cabeçalhos ao redor dos dados. Depois, à medida que subíamos para camadas mais altas, removemos camadas dos dados.



Ao descer as camadas, cabeçalhos são adicionados; ao subir, são removidos.

Cada camada só precisa entender seu próprio cabeçalho, e está “se comunicando” (em certo sentido) com seus pares na mesma camada. Quando a Secretária A escreve o nome no envelope, isso é destinado à Secretária B ler (não aos carteiros, nem ao chefe).

De forma mais formal, na Internet, os pares na mesma camada se comunicam estabelecendo um protocolo naquela camada. O protocolo só faz sentido para as entidades daquela camada específica.



Pares na mesma camada se comunicam por meio de um protocolo comum.

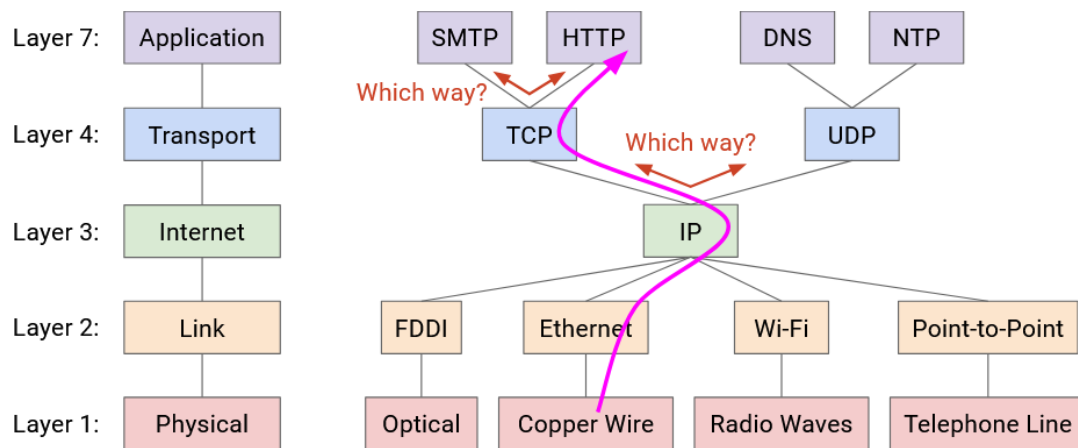
Neste projeto, nosso foco principal serão os cabeçalhos da Camada 3 (Internet) e da Camada 4 (Transporte).

Isso significa que nossos pacotes terão um cabeçalho externo, de camada mais baixa (por exemplo, Camada 3), e um cabeçalho interno, de camada mais alta (por exemplo, Camada 4).

1.7. Demultiplexação

Desde que o remetente e o destinatário concordem com a pilha de protocolos sendo usada, eles podem escolher qualquer pilha de protocolos que quiserem (geralmente um por camada).

Quando o destinatário recebe o pacote, ele precisa desembulhar os cabeçalhos, passando o pacote para cima através dos protocolos de camadas superiores.

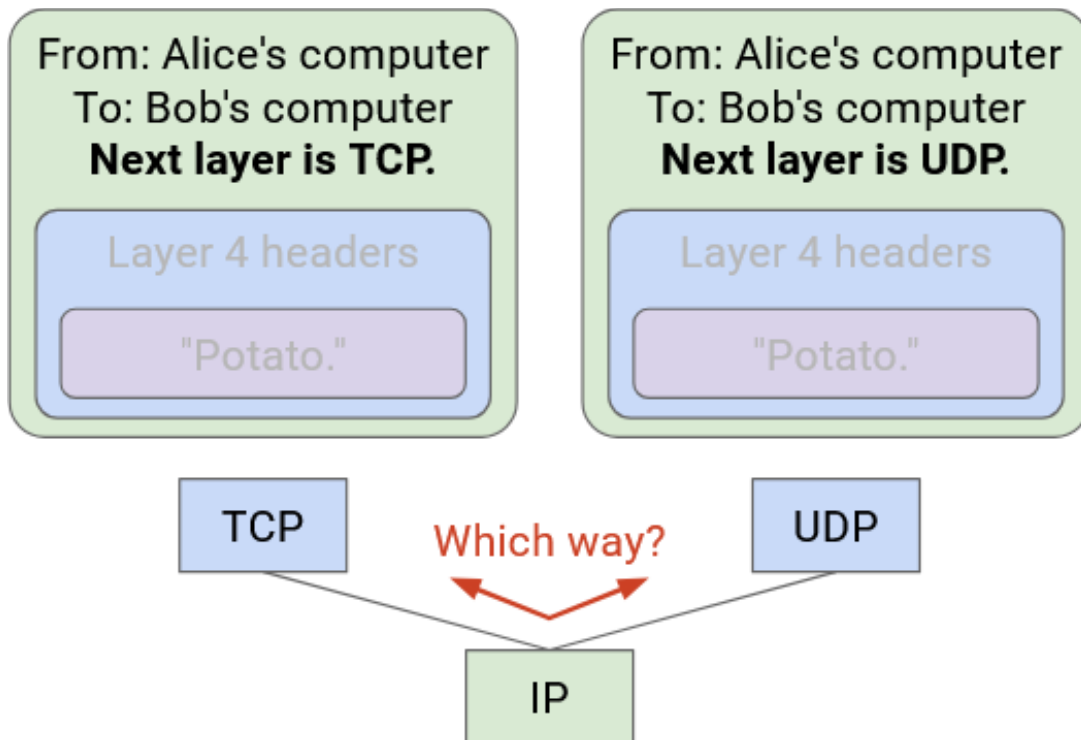


O destinatário desembulha os cabeçalhos, passando o pacote para cima pelas camadas.

Como o IP sabia que devia passar o pacote para o código do TCP, e não para o do UDP?

Como o TCP sabia que devia passar o pacote para o código do HTTP, e não para o do SMTP?

Para resolver esse problema, vamos adicionar um campo de cabeçalho adicional para **demultiplexação**. Esse campo de cabeçalho nos diz qual é o próximo protocolo de camada superior. Em outras palavras, ele nos diz qual protocolo devemos usar para interpretar o restante do payload, depois de retirarmos o cabeçalho da nossa camada.



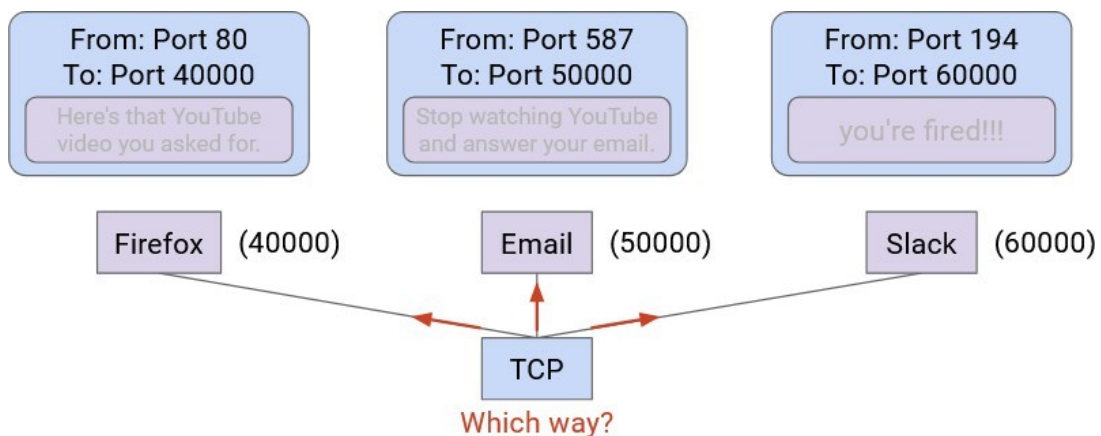
O campo de demultiplexação indica qual protocolo de camada superior processar em seguida.

1.8. Demultiplexação com Portas

A demultiplexação também funciona na Camada 4.

Cada aplicação em execução no seu computador está associada a um **número de porta**.

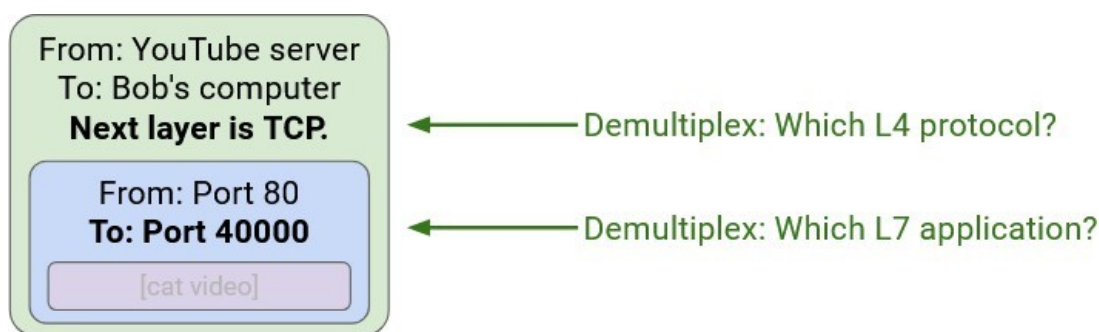
Quando o protocolo de Camada 4 recebe um pacote, ele usa o número da porta para passar o pacote para a aplicação correspondente.



O número de porta identifica a aplicação de destino na Camada 4.

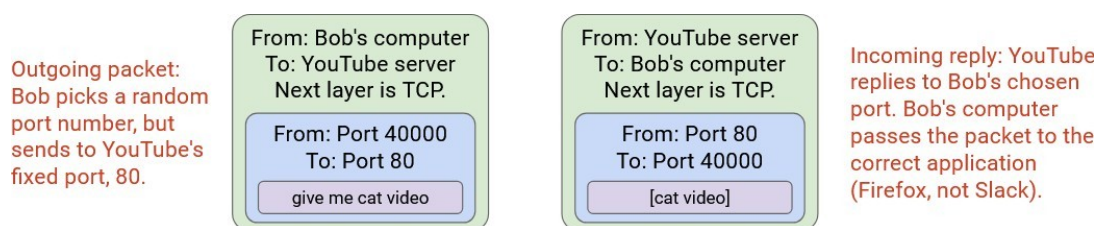
Na analogia postal: imagine se você e seu colega de casa têm o mesmo endereço. Se alguém enviar uma carta para sua casa, ela é para você ou para seu colega? Podemos diferenciar atribuindo números de quarto a cada morador.

Note que a demultiplexação pode funcionar tanto na Camada 3 quanto na Camada 4. No cabeçalho externo da Camada 3, escrevemos qual protocolo de Camada 4 vem em seguida. Depois, no cabeçalho interno da Camada 4, escrevemos qual aplicação vem em seguida.



A demultiplexação ocorre tanto na Camada 3 (protocolo) quanto na Camada 4 (aplicação/porta).

Note que, se duas aplicações estão trocando mensagens de ida e volta, ambas as aplicações têm um número de porta.



Duas aplicações trocando mensagens, cada uma com seu próprio número de porta.

Lembre-se de que a Camada 4 pode ser usada para implementar recursos diferentes, como demultiplexação e confiabilidade. Protocolos diferentes oferecem recursos diferentes.

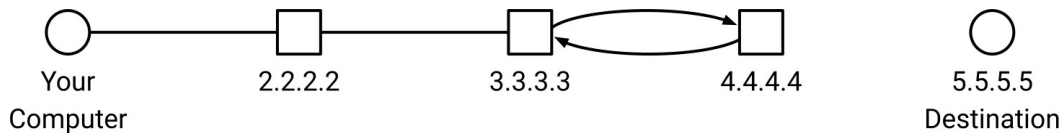
Neste projeto, usaremos o **UDP** (User Datagram Protocol) como nosso protocolo de Camada 4. O UDP oferece demultiplexação com portas, mas não oferece confiabilidade.

Aviso: em redes, existem duas coisas diferentes, ambas chamadas de “portas”. Acabamos de ver as *portas lógicas*, que são números usados para identificar aplicações em software. Há um conflito de nomenclatura com as *portas físicas*, que são os buracos onde você conecta um cabo. Neste projeto, trataremos principalmente de portas lógicas (e esclareceremos se algo não estiver claro).

2. Introduzindo o Traceroute

2.1. Time-to-Live (TTL)

Roteadores na Internet podem ter bugs.



Topologia: do seu computador até 1.1.1.1, 2.2.2.2, 3.3.3.3, 4.4.4.4 e de volta a 3.3.3.3, criando um laço entre 3.3.3.3 e 4.4.4.4. O destino (5.5.5.5) nunca é alcançado.

Suponha que você queira enviar pacotes para 5.5.5.5.

Você encaminha o pacote para 2.2.2.2.

Então, 2.2.2.2 decide encaminhar o pacote para 3.3.3.3.

Agora, imagine que 3.3.3.3 e 4.4.4.4 são roteadores com bugs. 3.3.3.3 envia o pacote para 4.4.4.4, que envia o pacote de volta para 3.3.3.3, que envia o pacote de volta para 4.4.4.4, e assim por diante. Nosso pacote está preso em um laço infinito!

Seria um desperdício de recursos se 3.3.3.3 e 4.4.4.4 continuassem encaminhando o mesmo pacote para sempre.

Para resolver esse problema, adicionaremos um cabeçalho extra no cabeçalho de Camada 3 (IP), chamado **time-to-live (TTL)**, ou “tempo de vida”. O TTL impõe um número máximo de saltos que um pacote pode dar antes de expirar.

Quando um roteador recebe um pacote, ele primeiro decrementa o TTL (modificando o cabeçalho) antes de encaminhar o pacote.

Se você receber um pacote com TTL igual a 1, você vai modificar o cabeçalho e chegar a um TTL de 0. Isso significa que o pacote expirou, então você deve descartá-lo (ignorá-lo e não encaminhá-lo).

Quando um roteador recebe um pacote expirado, ele também deve enviar uma mensagem de erro de volta ao remetente original, para que o remetente saiba que seu pacote foi descartado.

2.2. Explorando o TTL

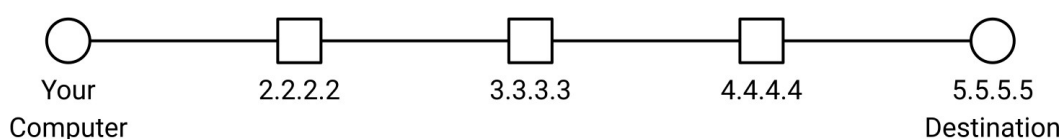
Normalmente, se você envia um pacote e recebe um pacote de resposta, você não sabe qual caminho os pacotes usaram para viajar pela rede. Em outras palavras, você não consegue descobrir quais roteadores intermediários encaminharam seu pacote. Nenhum dos campos de cabeçalho te conta sobre o caminho que o pacote usou.

Consegue pensar em uma forma de explorar o campo TTL para descobrir os roteadores ao longo do caminho entre você e um destino específico?

A ideia principal é: se ajustarmos o TTL baixo o suficiente, o pacote vai expirar antes de chegar ao destino. Quando o pacote expira, o roteador vai te enviar uma mensagem de erro. O cabeçalho da mensagem de erro dirá: “Do roteador, para você.” Isso permite que você descubra um roteador ao longo do caminho até o destino!

O utilitário **tracert** tenta descobrir todos os roteadores ao longo do caminho entre você e um destino especificado, enviando pacotes com TTLs sucessivamente maiores.

Em um nível alto, o traceroute funciona assim:



Envie um pacote para o destino, com TTL 1. Isso vai chegar ao primeiro roteador e expirar ali. O primeiro roteador (1.1.1.1) vai te enviar uma mensagem de erro, o que permite descobrir o primeiro roteador.

Envie um pacote para o destino, com TTL 2. Isso vai chegar ao segundo roteador e expirar ali. O segundo roteador (2.2.2.2) vai te enviar uma mensagem de erro, o que permite descobrir o segundo roteador.

Envie um pacote para o destino, com TTL 3. Isso vai chegar ao terceiro roteador (3.3.3.3) e expirar ali. O terceiro roteador vai te enviar uma mensagem de erro, o que permite descobrir o terceiro roteador.

Note que todo pacote que você envia é destinado ao destino especificado. Você nunca envia pacotes destinados aos roteadores intermediários (já que o objetivo do traceroute é justamente você não saber quem são esses roteadores).

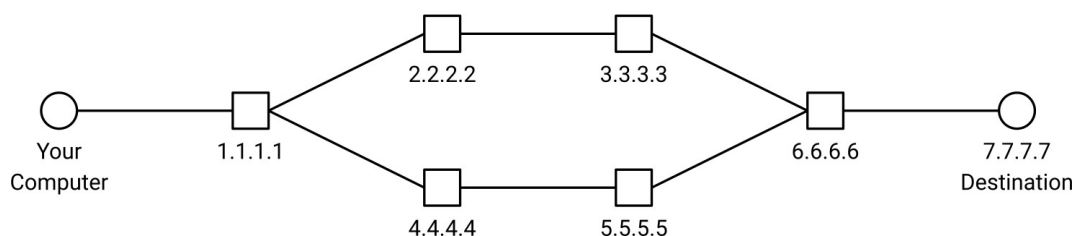
Os pacotes que enviamos são pacotes UDP-sobre-IP. Já as mensagens de erro dos roteadores são pacotes ICMP-sobre-IP. O protocolo ICMP tem uma estrutura de cabeçalho diferente, que permite ao roteador codificar o tipo específico de erro no cabeçalho (nesse caso, o erro é “TTL Excedido”).

Observação: na vida real, nem todos os roteadores têm garantia de enviar uma mensagem de erro de volta. O traceroute só precisa imprimir todos os roteadores que descobrir (mesmo que alguns roteadores do caminho não tenham sido descobertos).

2.3: Sondagem Repetida

Existem dois pequenos problemas com nosso design de traceroute até agora.

1. A Internet é de melhor esforço, e o protocolo de Camada 4 que escolhemos (UDP) não oferece confiabilidade. Isso significa que os pacotes que estamos enviando podem ser descartados, e os pacotes de resposta dos roteadores também podem se perder.
2. E se houver múltiplos caminhos entre você e o destino específico? Pode haver dois roteadores que estão ambos a 3 saltos de distância de você. Idealmente, gostaríamos de descobrir os roteadores ao longo de todos os caminhos.



Topologia com dois caminhos: do seu computador até 1.1.1.1, e então por 2.2.2.2/3.3.3.3 ou por 4.4.4.4/5.5.5.5, convergindo em 6.6.6.6 até o destino 7.7.7.7.

Para resolver ambos os problemas, em vez de enviar apenas um pacote para cada TTL, vamos enviar múltiplos pacotes para cada TTL.

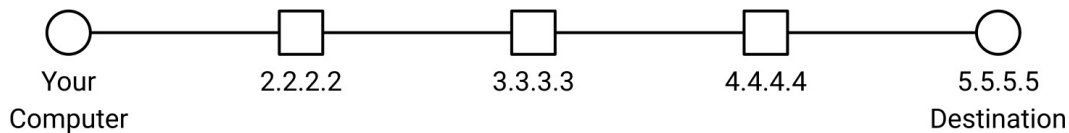
A propósito, cada pacote que enviamos durante o traceroute às vezes é chamado de **sonda** (probe). Enviar múltiplas sondas ajuda a mitigar ambos os problemas!

1. Sondas e respostas ainda podem ser descartadas, mas agora estamos enviando múltiplas sondas, então aumentamos a chance de pelo menos uma sonda/resposta ter sucesso.
2. Se enviarmos múltiplas sondas, podemos tentar atingir todos os roteadores a uma certa quantidade de saltos de distância.

Note que, na vida real, não há garantia de que descubramos todos os roteadores a uma dada distância das nossas sondas. Por exemplo, todas as sondas com TTL 3 poderiam seguir o caminho de cima, de modo que só descobríssemos 3.3.3.3, mas não 5.5.5.5. Nos cenários de teste do projeto, a rede simulada é determinística e distribui as sondas de modo que todos os roteadores previstos pelo cenário possam ser observados.

2.4: Portas Inalcançáveis

Como sabemos quando terminamos de executar o traceroute? Considere o exemplo de antes:



Topologia: do seu computador até 2.2.2.2, 3.3.3.3, 4.4.4.4 e 5.5.5.5 (destino).

Quando você envia um pacote com TTL 4, ele vai chegar ao destino.

Se o destino não responder, não vamos saber que terminamos de traçar um caminho até ele. Continuaríamos enviando pacotes com TTL 5, TTL 6, TTL 7, etc. e não obteríamos respostas.

Para convencer o destino a nos enviar uma resposta, vamos intencionalmente enviar o pacote para uma **porta inalcançável** no destino.

Lembre-se de que, em um computador, cada serviço em execução está associado a um número de porta. Se escolhermos um número de porta que não está associado a nenhum serviço em execução, o computador de destino vai ficar confuso e enviar de volta uma mensagem de erro “Porta Inalcançável”: “A porta que você solicitou não corresponde a nenhuma aplicação existente.”

A mensagem “Porta Inalcançável” é enviada como um pacote ICMP-sobre-IP. Lembre-se: o protocolo ICMP tem uma estrutura de cabeçalho diferente, incluindo um código de erro (nesse caso, o erro é “Porta Inalcançável”).

Observação: esse comportamento depende muito da configuração da máquina de destino. Nos cenários de teste, o destino simulado responde com a mensagem ICMP de Porta Inalcançável quando a sonda chega ao host final. Em redes reais, algumas máquinas podem não enviar essa mensagem.

Observação: não existe um padrão formal para isso, mas muitos computadores são configurados para nunca usar o número de porta 33434, para que o traceroute possa usar essa porta para disparar mensagens de Porta Inalcançável.

3. Estrutura do Cabeçalho

Neste projeto, ao implementar o traceroute, você precisará enviar pacotes, preenchendo todos os campos de cabeçalho necessários.

Você também precisará receber pacotes e processar os campos de cabeçalho relevantes.

Nesta seção, vamos destacar os campos de cabeçalho relevantes que você precisa conhecer para este projeto.

3.1: Campos do Cabeçalho IP

Campo	Tamanho (bits)	Descrição
Versão	4	Versão do protocolo IP sendo usado. Para este projeto, sempre usamos IPv4 (versão 4).
Tamanho do Cabeçalho	4	Tamanho do cabeçalho IP. Medido em palavras de 4 bytes. (Isso é necessário por causa das opções de tamanho variável no final).
Tipo de Serviço	8	Usado para marcar pacotes como de alta prioridade. Não é necessário para este projeto.
Tamanho Total do Pacote	16	Tamanho do pacote inteiro (cabeçalho e payload). Medido em bytes.
Identificação	16	Usado para fragmentação. Não é necessário para este projeto.
Flags	3	Usado para fragmentação. Não é necessário para este projeto.
Offset	13	Usado para fragmentação. Não é necessário para este projeto.
TTL	8	Número de saltos restantes antes que este pacote expire.
Protocolo	8	Para qual protocolo de camada superior devemos passar o payload do IP? Códigos relevantes para este projeto: 1 = ICMP. 17 = UDP.
Checksum	16	Verifica que o pacote não foi corrompido em trânsito. Para este projeto, você pode ignorar checksums: todos os pacotes de saída terão esse campo definido como 0 (indicando que nenhum checksum foi calculado). Você não precisa verificar checksums em pacotes recebidos.
Endereço de Origem	32	Endereço IP da máquina que envia o pacote.
Endereço de Destino	32	Endereço IP da máquina de destino.

Campo	Tamanho (bits)	Descrição
Opções	Variável	Campo de tamanho variável para solicitar funcionalidades avançadas. Se nenhum processamento especial for necessário, esse campo fica vazio (0 bytes). Para este projeto, você pode (majoritariamente) ignorar as opções: todos os pacotes de saída não terão esse campo. Se um pacote recebido tiver Opções, você pode ignorá-las.

3.2: Campos do Cabeçalho UDP

Campo	Tamanho (bits)	Descrição
Porta de Origem	16	Número da porta da aplicação que está enviando este pacote.
Porta de Destino	16	Número da porta da aplicação que estamos contatando no destino.
Tamanho do Pacote	16	Tamanho do pacote inteiro (cabeçalho e payload). Medido em bytes.
Checksum	16	Verifica que o pacote não foi corrompido em trânsito. Para este projeto, você pode ignorar checksums.

3.3: Campos do Cabeçalho ICMP

Campo	Tamanho (bits)	Descrição
Tipo	8	Tipo de erro. Códigos relevantes para este projeto: 3 = Destino Inalcançável. 11 = Tempo Excedido.
Código	8	Código de erro. Códigos relevantes para este projeto: 3 = Porta de Destino Inalcançável. 0 = TTL Expirado.
Checksum	16	Verifica que o pacote não foi corrompido em trânsito. Para este projeto, você pode ignorar checksums.
Resto do Cabeçalho	32	O conteúdo varia de acordo com o tipo e código do ICMP. Para os erros ICMP deste projeto, esses bits são sempre definidos como 0.

O payload do ICMP contém uma cópia do cabeçalho do pacote original (UDP-sobre-IPv4) que disparou o erro.

Materiais de Referência Sugeridos

Traceroute:

- Traceroute na Wikipédia: en.wikipedia.org/wiki/Traceroute
- Página de manual do BSD Traceroute: man.freebsd.org/cgi/man.cgi?query=traceroute
- Página de manual do Linux Traceroute: linux.die.net/man/8/traceroute

Protocolos:

- IPv4 na Wikipédia (estrutura do pacote):
en.wikipedia.org/wiki/Internet_Protocol_version_4#Packet_structure
- UDP na Wikipédia (estrutura do datagrama UDP):
en.wikipedia.org/wiki/User_Datagram_Protocol#UDP_datagram_structure
- ICMP na Wikipédia (estrutura do datagrama):
en.wikipedia.org/wiki/Internet_Control_Message_Protocol#Datagram_structure
- RFC do IP: datatracker.ietf.org/doc/html/rfc791
- RFC do UDP: datatracker.ietf.org/doc/html/rfc768
- RFC do ICMP: datatracker.ietf.org/doc/html/rfc792